

ROSETTA SECRET COMMUNICATIONS

System Design, Implementation & Optimization

I. Quick Introduction

This has been an incredible solo project for me. I started it as an early-intermediate C systems developer on GNU/Linux. Having undertaken it has taught me colossal amounts in topics spanning from the basics, such as the dangers of feature creep and the importance of checking for and handling errors in GNU/Linux syscalls, to the intermediate, such as writing easy-to-maintain Makefiles, to advanced, such as compiler optimizations & ways a programmer can influence them positively in source code by (correctly) using compiler hints like `__restrict__` and refactors that open up the compiler's eyes when deciding which of its own optimizations to apply, as well as features of the C language whose purpose doesn't become apparent until one has written a system that's way beyond trivial in size, like function pointers. It continues to be a valuable platform where I can practice advanced systems programming.

In its essence, Rosetta is a system that allows secret communication.

I have implemented the following 5 main components completely from scratch using only the C language, its standard library, GNU/Linux system calls and standard GCC compiler features, without external dependencies on any libraries and importantly - without using vibe coding or "AI"-generated code at all, as I have concluded firmly that this practice, when over-used to generate code that is then blindly added to a non-trivial engineering project as long as it appears to work (it more often than not doesn't) DIMINISHES an engineer's skills.

1. Big Integer math engine: From ADD/SUB to Montgomery Multiplication.
2. Cryptography engine: chacha20, blake2b, argon2, signatures and more.
3. Communication server
4. Communication client
5. Test Framework, simulating real human users with local OS processes.

In the end, it came out to be **roughly 10,000 lines of C code**.

Rosetta is designed to be highly modular:

The BigInt math & cryptography engines can be used for other purposes.

The communication client and server are designed to allow any number of communication mechanisms to be easily used in the system, as long as said mechanism implements 4 basic functions - begin communication, send payload, receive payload and end communication. Such elegance and modularity is possible thanks to a polymorphic C interface, whose only requirement is that, at system initialization, 4 function pointers are set to the respective C or C++ functions that, in turn, implement the 4 respective communication tasks. Currently two communication interfaces are supported: 1) Internet communication over TCP using Internet sockets and 2) local OS processes sending each other messages via Unix Domain sockets, to simulate human users in the Rosetta Test Framework. A future possible addition here could be radio communication where the client runs on a custom embedded communicator device.

Lastly on modularity, the client allows for multiple different "client driver" programs to be written, thus providing different ways to use Rosetta. They simply need to use the API exposed by the Client Primary Functions collection found in the header file under the same name and the client driver program can then wrap these in higher-level end-user facing functionality, where the users can be people texting each other over the internet or just as easily - a set of machines that must automatically deliver messages to each other in secret. This is how, as mentioned above, radio messaging using a custom embedded communicator device could be implemented in the future. So far, I have two client driver programs - a human user-facing C++ wxWidgets GUI that supports the creation and joining of chat rooms and secondly, the set of manual and automatic user-spawning programs for whole-system test simulations in the Rosetta Test Framework.

Wrapping up the introduction, I give an overview of the security scheme that the Rosetta system uses and implements. When reading it, please note that the parameters, such as bit size of prime numbers and of cryptography artifacts like salts and nonces, can easily be updated in the future, should security concerns dictate doing so.

Rosetta Security Scheme overview

Everything begins with 2 big prime numbers: 3072-bit M - the Diffie-Hellman modulus - and 320-bit prime Q , where $(M-1)$ is divisible by Q .

Rosetta comes with a utility program to discover such pairs of primes, using Rabin-Miller primality test provided by my BigInt math engine. Once we have M and Q, we compute the Diffie-Hellman generator G:

$$G = 2^{(M-1)/Q} \bmod M$$

Now the server generates its long-term private/public key pair (b,B). Private key has the same bitwidth as Q and is strictly smaller than Q:

- Private key b = 320 pseudorandom bits taken from /dev/urandom, $b < Q$.
- Public key $B = G^b \bmod M$

Server's long-term public key B, its Montgomery form Bm, as well as M, Q, G and G's Montgomery form Gm are all available to clients at install.

Internet connection is NOT needed for a client to create a local savefile. This is called **Registration**. The user picks a password. The client software generates the user's long-term private/public key pair (a,A) exactly like the server does it. The password is used as a key in Argon2id to produce a 64-byte hash T. The most significant 32 bytes of hash T are then used as a key in ChaCha20 to encrypt the user's private key. The ChaCha20 Nonce and Argon2id Salt are stored on the client machine in the savefile, along with the user's public key and encrypted private key.

A user must **log in** before joining a chat room to talk to others in secret. For this, the user enters their password, it's analogously used in Argon2 to produce hash T, the most significant 32 bytes of that hash are used in ChaCha20 to decrypt the user's private key. A public key is computed from that private key. If it matches the stored public key, the password was correct and the login proceeds to connect to the Rosetta server.

The client sends their long-term public key to the server in encrypted form using a very short-lived ephemeral public/private key pair that was generated by both the server and the client to produce a short-lived DH shared secret just for that purpose. Short-lived unidirectional session keys KAB/KBA are extracted from the shared secret and used to encrypt/decrypt communication until the server has the client's long-term public key. Once the server obtains & decrypts the client long-term public key, the short-lived key pairs & shared secret are destroyed. HMAC authentication is used before the server has the client's long-term public key. Schnorr Signatures are used after that.

When Alice and Bob meet in a chat room, regardless of whether they've met before, they always first perform a handshake. The server first sends them the other side's public key. They compute a shared secret X and extract unidirectional session keys K_{AB} and K_{BA} , as well as a chacha Nonce from the shared secret:

1. Alice's client computes a session-length shared secret: $X = B^a \bmod M$
Bob's client computes the same session shared secret: $X = A^b \bmod M$

2. On Alice's side:
 K_{AB} = least significant 32 bytes of X
 K_{BA} = next 32 bytes of X .

On Bob's side:
 K_{BA} = least significant 32 bytes of X
 K_{AB} = next 32 bytes of X .

3. On Alice's side, swap K_{AB} with K_{BA} if $A < B$.
On Bob's side, swap K_{BA} with K_{AB} if $A > B$.

This ensures K_{AB} and K_{BA} are the same on Alice's and Bob's sides.

Whether Alice uses session key K_{AB} to send to Bob and Bob uses K_{AB} to receive from Alice and then K_{BA} is used in the other direction, or the other way around, is decided by asking "who was in the chat room first?" Alice and Bob each have 2 symmetric ChaCha20 Nonce counters, one for Alice sending to Bob and one for Bob sending to Alice. They are incremented on each usage of ChaCha20 because re-using the same Nonce with the same ChaCha20 key is forbidden.

When Alice decides to send a secret message to Bob, Carol and Fred as they are all in the same chat room, she generates a pseudorandom one time use key K for each message. Alice encrypts one-time-use key K with chacha20, using the unidirectional session key K_{AB}/K_{BA} she has with all other guests (from her Diffie-Hellman shared secret that she has with everyone). This produces 3 encrypted versions of the one-time-use key K - K_B , K_C and K_F . Now Alice uses plaintext version of one time use key K to encrypt the secret payload itself. Lastly, she generates a Schnorr Signature on the payload and sends the encrypted message, the encrypted keys used to decrypt the message (for each person) and her signature.

So the secret message is hidden behind not one, but two keys.

To compute a Schnorr Signature, the steps are:

0. Call BLAKE2B{64} with input - whatever we're signing.
The produced 64-byte hash of this is called the prehash PH.
1. Call BLAKE2B{64} with input - the signer's private key concatenated with the prehash, reduce the result of this modulo (Q-1), add 1.

Yields secret: $k = (\text{BLAKE2B}\{64\}(a || \text{PH}) \bmod (Q-1)) + 1$

2. Compute $R = G^k \bmod M$
3. Compute $e = \text{BLAKE2B}\{64\}(R || \text{PH})$, truncated to bitwidth of Q.
4. Compute $s = (k + ((Q - a) \times e)) \bmod Q$

The signature itself is (s,e)

To verify against public key A and whatever was signed, the receiver:

0. checks that $0 \leq s < Q$ and that e has the bitwidth of Q.
1. Computes the prehash PH as in step 0 above.
2. Computes $R = (G^s * A^e) \bmod M$.
3. Computes $\text{val_e} = \text{BLAKE2B}\{64\}(R || \text{PH})$, truncated to bitwidth of Q.
Check that this is equal to e. If it is, validation passed.
Under any other circumstances, validation fails.

In the end, when Bob receives Alice's secret message (which was relayed by the Rosetta server which itself computed its own Schnorr signature so the receiver can authenticate it too), he first validates the server's signature, so he is assured that the message really was relayed by the real Rosetta server and was not altered en-route. Then he validates Alice's own signature, assuring himself that the message really was sent by her and wasn't altered en-route. He uses the same unidirectional session key KAB/KBA (extracted from his shared secret with Alice) to decrypt the one-time use key K, then he uses the decrypted key K to actually decrypt the secret message itself and read it.